



EGE UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

2024–2025 FALL SEMESTER

INTRODUCTION TO DATABASES
E-COMMERCE PROJECT

DELIVERY DATE

17.01.2025

PREPARED BY

TEAM NO : 22

05220000956 Murat Güneş Özkara 05220000303 Burak Kocabaş

05220000294 Yusuf Enes Sözeri 05220000993 İsa Selim Balcılar

TRENDYOL

For customers, the database stores customer name, email, phone number, date of registration, address details including street, city, state, postal code, and country. Customers can create their own reviews which may include comments, ratings, and review date. Each customer maintains a cart that has products with their quantities and prices, also tracking the total price for all products. Multiple coupons can be applied to carts. The system keeps track of customer payment preferences including payment name and card details. Customers can follow shops. Customers can be elite member; it has no payment for this elite member's gain elite point from reviews and orders.

For products, the database maintains detailed information including title, price, stock quantity, description, and SKU. Products can be sold through different shops, and each product tracks its seller information including shop name, shop description, and seller rating. Products can have variants with specific details and prices. Products are organized into collections under specific topics, and the system tracks related products for recommendations. Each product maintains its rating statistics and review counts.

For orders, the system tracks order date, status (pending, shipped, delivered), shipping details, and total price. Orders are linked to specific products with their quantities and prices at the time of purchase. The system maintains order items which track individual product details within each order. Customers can have multiple active orders, and each order must be linked to valid payment information. Each item ordered can have specific shipment information.

For coupons, the database stores coupon codes, their discount types, valid date ranges, and usage limits. Coupons can be applied to specific products or categories, and the system tracks how many times each coupon has been used. Coupons must be provided by the shop for own shop items, and the system validates coupon eligibility during checkout.

The shop system maintains shop profiles with shop names, descriptions, and contact details. Each shop can have multiple products, and their products can be organized into collections. The system tracks shop ratings and review statistics. Shops can organize their products into different categories and manage their own inventory levels.

For collections, the system manages groupings of products under specific themes or topics. Collections have names, descriptions, and can contain multiple products. The database tracks which products belong to which collections, allowing for dynamic organization of the product catalog.

The database supports efficient querying for order history, product availability, and customer information while maintaining security of sensitive data.

Customers add their own address to the system and the system has also pick up point address for delivery location. Customers can select one address to order delivery address. For addresses, the system maintains detailed location information including street address, city, state/province, postal code, and country.

HEPSIBURADA

For customers, the database stores customer name, email, phone number, date of registration, address details including street, city, state, postal code, and country. Customers can create their own reviews which may include comments, ratings, and review date. Each customer maintains a cart that has products with their quantities and prices, also tracking the total price for all products. Just one coupon can be applied to carts. The system keeps track of customer payment preferences including payment name and card details and payment method. Customers can follow shops and also follows to customers. Customers can buy premium membership for some benefits

For products, the database maintains detailed information including title, price, stock quantity, description, and SKU. Products can be sold through different shops, and each product tracks its seller information including shop name, shop description, and seller rating. Products can have variants with specific details and prices. Products are organized into collections under specific topics, and the system tracks related products for recommendations. Each product maintains its rating statistics and review counts.

For orders, the system tracks order date, status (pending, shipped, delivered), shipping details, and total price. Orders are linked to specific products with their quantities and prices at the time of purchase. The system maintains order items which track individual product details within each order. Customers can have multiple active orders, and each order must be linked to valid payment information. Each item ordered can have specific shipment information.

For coupons, the database stores coupon codes, their discount types, valid date ranges, and usage limits. Coupons can be applied to specific products or categories, and the system tracks how many times each coupon has been used. Shops can supply coupon to customers for their own shop items also system can add coupon for different type product. They have not essential provided by shops, and the system validates coupon eligibility during checkout.

The shop system maintains shop profiles with shop names, descriptions, and contact details. Each shop can have multiple products, and their products can be organized into collections. The system tracks shop ratings and review statistics. Shops can organize their products into different categories and manage their own inventory levels.

For collections, the system manages groupings of products under specific themes or topics. Collections have names, descriptions, and can contain multiple products. The database tracks which products belong to which collections, allowing for dynamic organization of the product catalog.

The database supports efficient querying for order history, product availability, and customer information while maintaining security of sensitive data.

Customers add their own address to system and system has also pick up point address for delivery location. Customers can select one address to order delivery address. For addresses, the system maintains detailed location information including street address, city, state/province, postal code, and country.

AMAZON

For customers, the database stores customer name, email, phone number, date of registration, address details including street, city, state, postal code, and country. Customers can create their own reviews which may include comments, ratings, and review date. Each customer maintains a cart that has products with their quantities and prices, also tracking the total price for all products. Multiple coupons can be applied to carts. The system keeps track of customer payment preferences including payment name and card details. Customers can buy premium membership for some benefits.

For products, the database maintains detailed information including title, price, stock quantity, description, and SKU. Products can be sold through different shops, and each product tracks its seller information including shop name, shop description, and seller rating. Products can have variants with specific details and prices. Products are organized into collections under specific topics, and the system tracks related products for recommendations. Each product maintains its rating statistics and review counts.

For orders, the system tracks order date, status (pending, shipped, delivered), shipping details, and total price. Orders are linked to specific products with their quantities and prices at the time of purchase. The system maintains order items which track individual product details within each order. Customers can have multiple active orders, and each order must be linked to valid payment information. Each item ordered can have specific shipment information.

For coupons, the database stores coupon codes, their discount types, valid date ranges, and usage limits. Coupons can be applied to specific products or categories, and the system tracks how many times each coupon has been used. The system provides coupons, and the system validates coupon eligibility during checkout.

The shop system maintains shop profiles with shop names, descriptions, and contact details. Each shop can have multiple products, and their products can be organized into collections. The system tracks shop ratings and review statistics. Shops can organize their products into different categories and manage their own inventory levels.

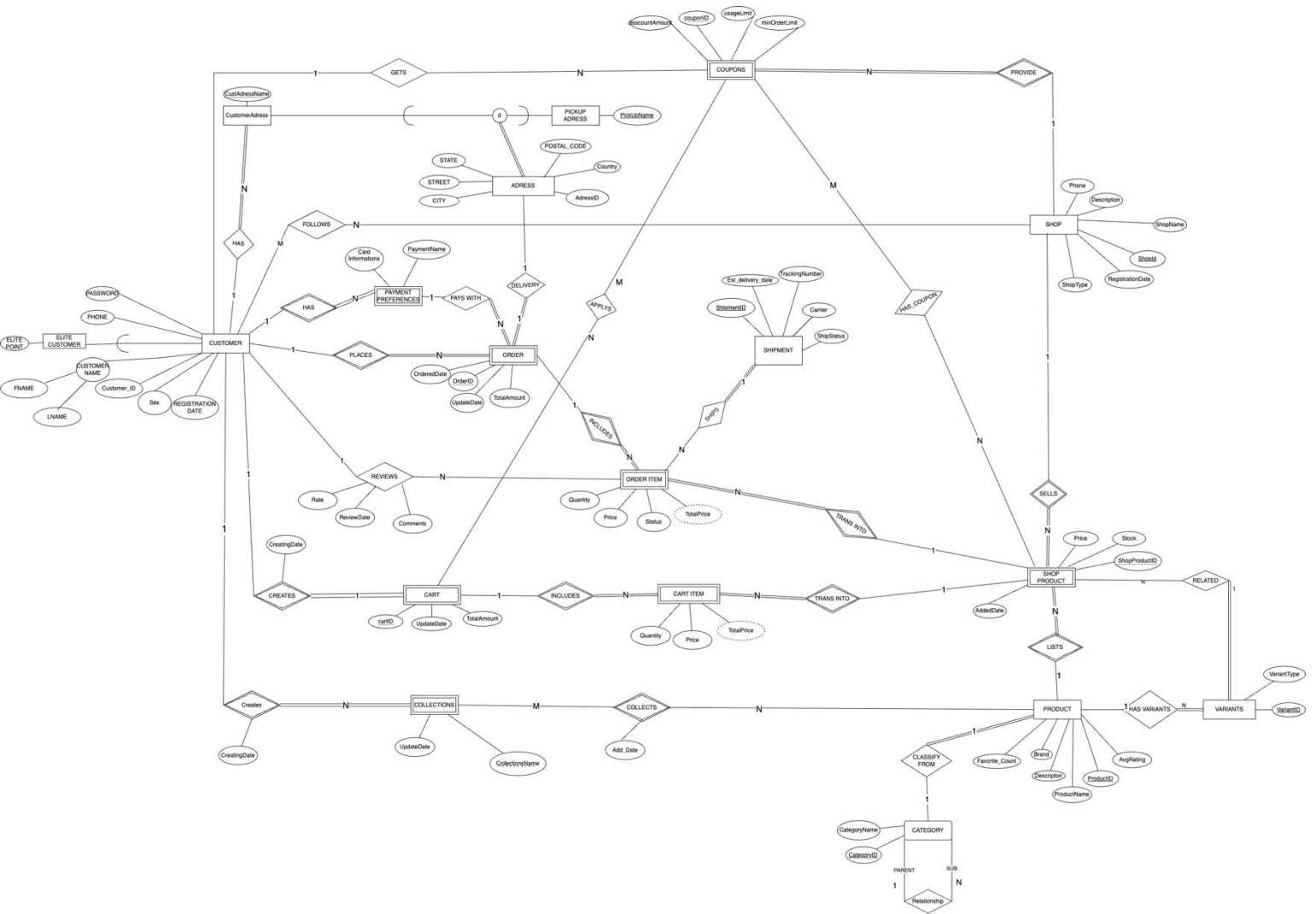
For collections, the system manages groupings of products under specific themes or topics. Collections have names, descriptions, and can contain multiple products. The database tracks which products belong to which collections, allowing for dynamic organization of the product catalog.

The database supports efficient querying for order history, product availability, and customer information while maintaining security of sensitive data.

Customers add their own address to the system and the system has also pick up point address for delivery location. Customer can send the products in an order to different addresses, and depending on the quantity of the product, even the same product can be sent to different addresses, the system maintains detailed location information including street address, city, state/province, postal code, and country.

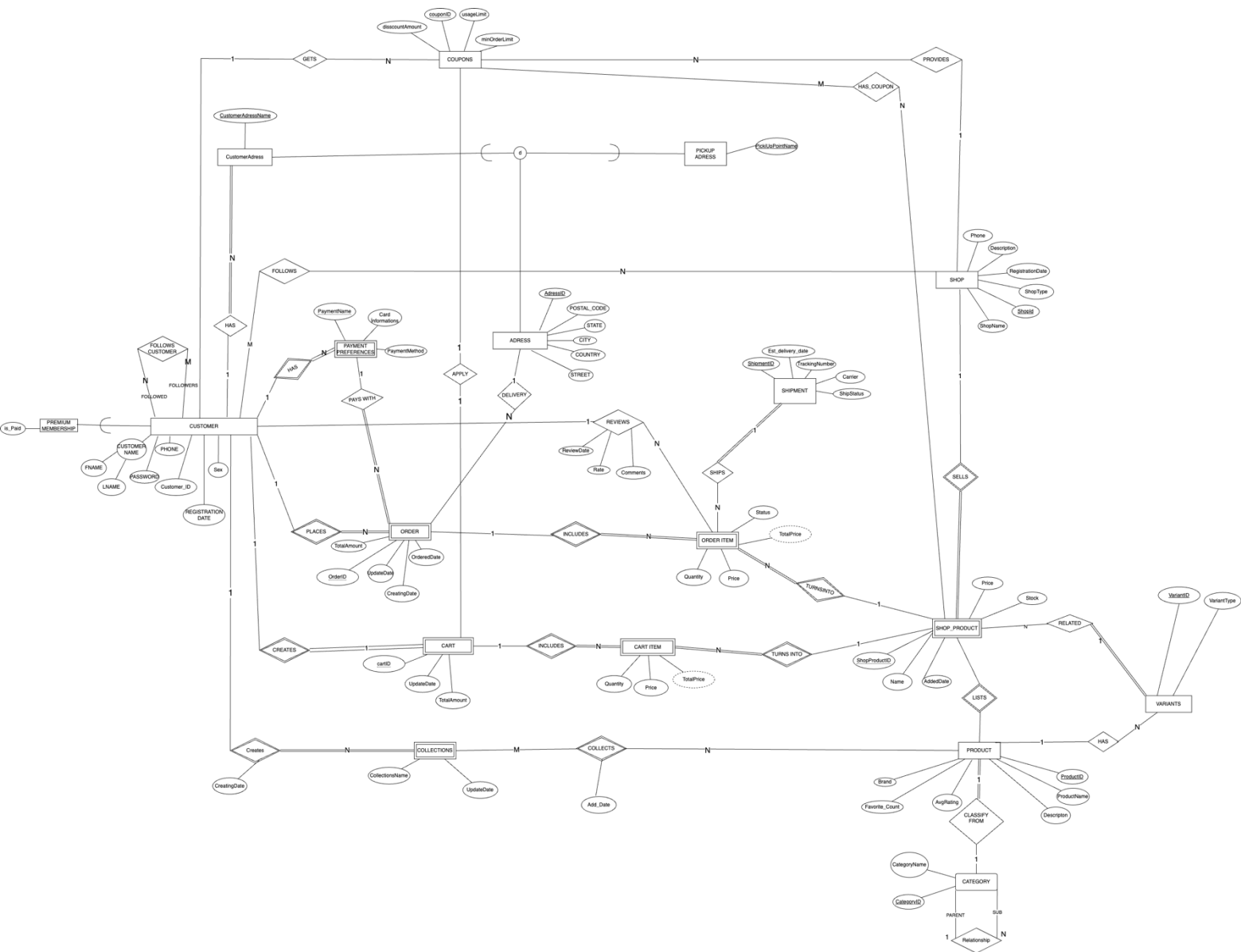
Amazon can stock the shops products in Amazon warehouses and provide the shipment, and the shops can also have their own warehouses. The system holds unique warehouse id and their locations.

TRENDYOL EER DIAGRAM



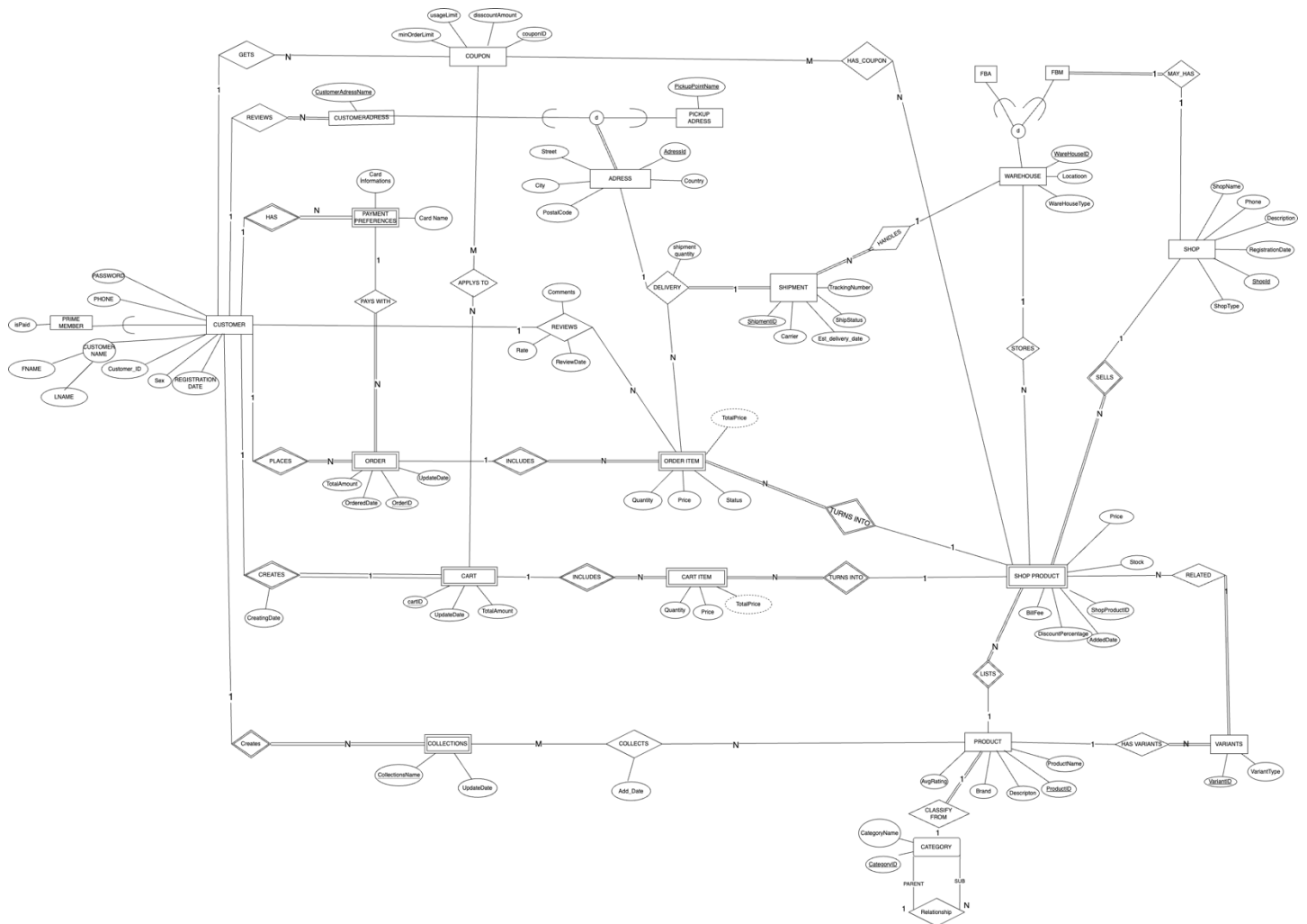
https://drive.google.com/file/d/1pVlfdL7kATyvYOe_lbdWEWb5zSuAK9hJ/view?usp=sharing

HEPSIBURADA EER DIAGRAM



<https://drive.google.com/file/d/1I5TIKYJMYCztjTFZSvDejnkNyTzUxGhA/view?usp=sharing>

AMAZON EER DIAGRAM

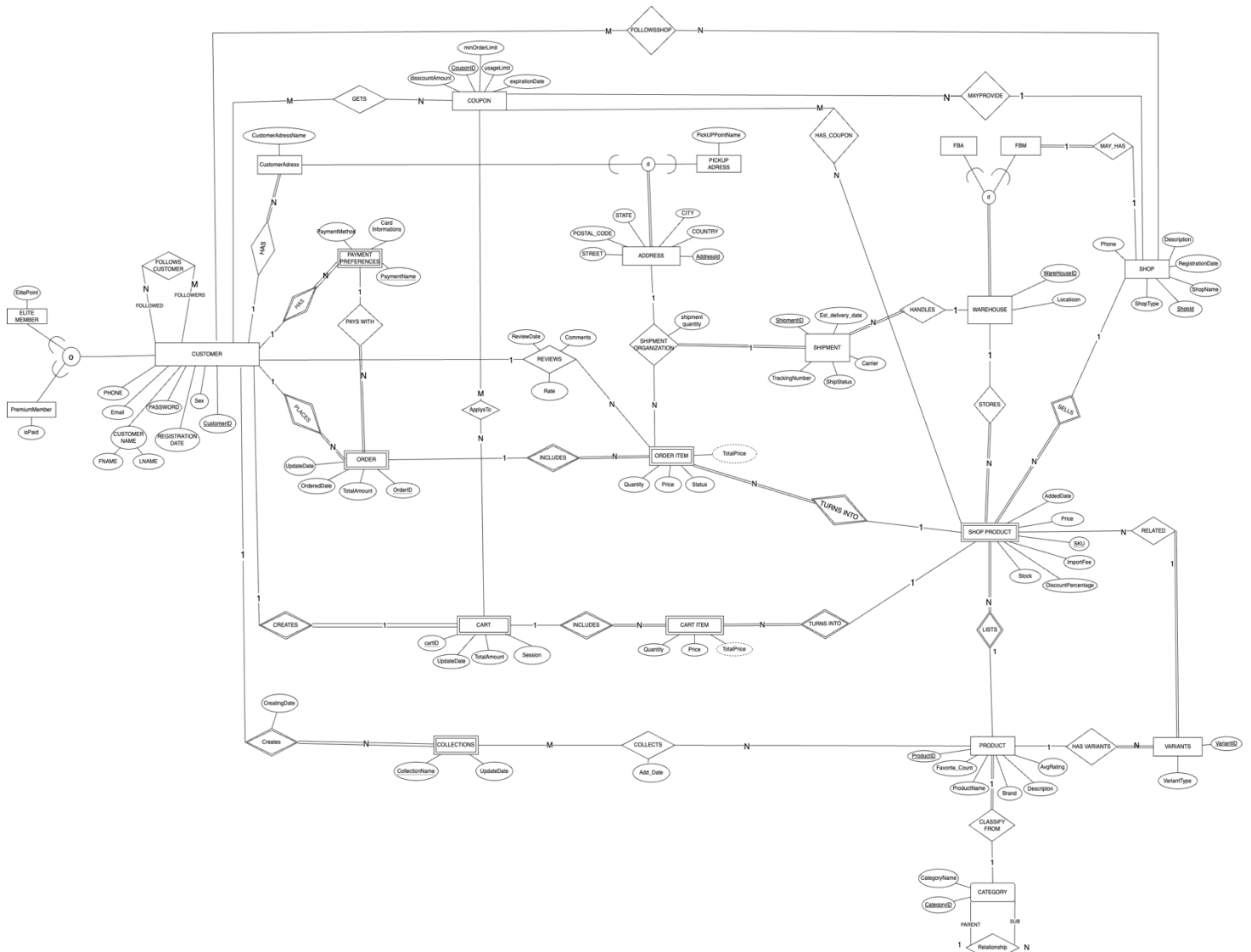


<https://drive.google.com/file/d/1XZW23m-rR-zfASIFfJIMAJV3jHHg-p0O/view?usp=sharing>

COMBINE EER METHODOLOGY

- 1) First of all, we added entities and relationships that are common in all diagrams together.**
- 2) Then we added the features that are unique for each diagram to the combined diagram in the same way as in the unique diagram.**
- 3) Combining situations in the same context**
 - a. In cases with the same context, the first thing we look at is whether the representation in one case covers the other, and if so, we add the most comprehensive case to the combined diagram.**
 - b. If there is no coverage and the states in the same context do not contradict each other when added separately to the system, we have added them separately to the new combined diagram according to the Open World Assumption.**
 - c. if there is no coverage and different ways of handling the same context in different diagrams create contradictions in the system when adding them to the system, we use Close World Assumptions to select the business logic in one of the diagrams and add it to our combined diagram.**

COMBINE EER DIAGRAM



https://drive.google.com/file/d/13TeBFOkMLI9DCP64RxgU4gkT_Hw_mkzF/view?usp=sharing

Iteration

Step 1

- CUSTOMER (CustomerID, Phone, Email, Password, Fname, Lname, RegistrationDate, Sex)
- ADRESS (AdressID, Country, State, City, Street, Postal_Code)
- SHIPMENT (ShipmentID, Est_Delivery_Date, TrackingNumber, ShipStatus, Carrier)
- WAREHOUSE (WarehouseID, Location)
- SHOP (ShopID, Phone, Description, RegistrationDate, ShopName, ShopType)
- PRODUCT (ProductID, Favorite_Count, ProcutName, Brand, Description)
- CATEGORY (CategoryID, CategoryName)
- COUPON (CouponID, disscountAmount, minOrderLimit, usageLimit, expirationDate)
- VARIANTS (VariantID, VariantType)

Step 2

- SHOPPRODUCT (sku, PRODUCT.ProductID, SHOP.ShopId, Price, Stock, DiscountPercentage, AddedDate)
- PAYMENTPREFERENCES (CUSTOMER.CustomerID, PaymentName, CardName, CardInformations)
- ORDER (CUSTOMER.CustomerID ,OrderID, TotalAmount, OrdererdDate, UpdateDate)
- CART (CUSTOMER.CustomerID, CartID, TotalAmount, Session, UpdateDate)
- COLLECTIONS (CUSTOMER.CustomerID, CollectionName, UpdateDate)

Step 3

- PRODUCT (ProductID, Favorite_Count, ProcutName, Brand, Description , CATEGORY.CategoryID)

Step 4

- VARIANTS (VariantID, VariantType, PRODUCT.ProductID)
- SHOPPRODUCT (sku, PRODUCT.ProductID, SHOP.ShopId, Price, Stock, ImportFee DiscountPercentage, AddedDate, VARIANT.VariantID, WAREHOUSE.WarehouseID)
- CATEGORY (CategoryID, CategoryName, ParentCategoryID)
- ORDER (CUSTOMER.CustomerID ,OrderID, TotalAmount, OrdererdDate, UpdateDate, PAYMENTPREFERENCES.PaymentName)

- SHIPMENT (ShipmentID, Est_Delivery_Date, TrackingNumber, ShipStatus, Carrier, WAREHOUSE.WarehouseID)
- COUPON (CouponID, discountAmount, minOrderLimit, usageLimit, expirationDate, SHOP.ShopID)

Step 5

- FOLLOWSHOPS (CUSTOMER.CustomerID, SHOP.ShopId)
- CUSTOMERCOUPON (CUSTOMER.CustomerID, COUPON.CouponID)
- FOLLOWCUSTOMER (CUSTOMER.CustomerID , CUSTOMER.CustomerID)
- HAS_COUPON (COUPON.CouponID, SHOPPRODUCT.SKU, PRODUCT.ProductID, SHOP.ShopId)
- CARTCOUPON (COUPON.CouponID, CUSTOMER.CustomerID, CART.CartID)
- COLLECTIONITEMS(COLLECTIONS.CollectionName, CUSTOMER.CustomerID, PRODUCT.ProductID)

Step 6

- -----

Step 7

- -----

Step 8

- CUSTOMER (CustomerID, Phone, Email, Password, Fname, Lname, RegistrationDate, Sex, EliteFlag, PremiumFlag, ElitePoint, IsPaid)
- WAREHOUSE (WarehouseID, Location ,WarehouseType)
- ADDRESS (AddressID, Country, State, City, Street, Postal_Code, AddressType, CustAddressName , PickUPPointName)

2nd Iteration

Step 1

- -----

Step 2

- ORDERITEM (CUSTOMER.CustomerID, ORDER.OrderID,
PRODUCT.ProductID, SHOP.ShopId, SHOPPRODUCT.SKU, Quantity, Price,
Status,)
- CARTITEM (CUSTOMER.CustomerID, CART.CartID, PRODUCT.ProductID,
SHOP.ShopId, SHOPPRODUCT.SKU, Quantity, Price, Status,)

Step 3

- WAREHOUSE (WarehouseID, Location ,WarehouseType, SHOP.ShopId)

Step 4

- REVIEWS (CUSTOMER.CustomerID, ORDER.OrderID,
PRODUCT.ProductID, SHOP.ShopId, SHOPPRODUCT.SKU, Rate,
Comment, ReviewDate)
- ADRESS (AdressID, Country, State, City, Street, Postal_Code, AddressType,
CustAdresName , PickUPPointName , CUSTOMER.CustomerID)

Step 5

- -----

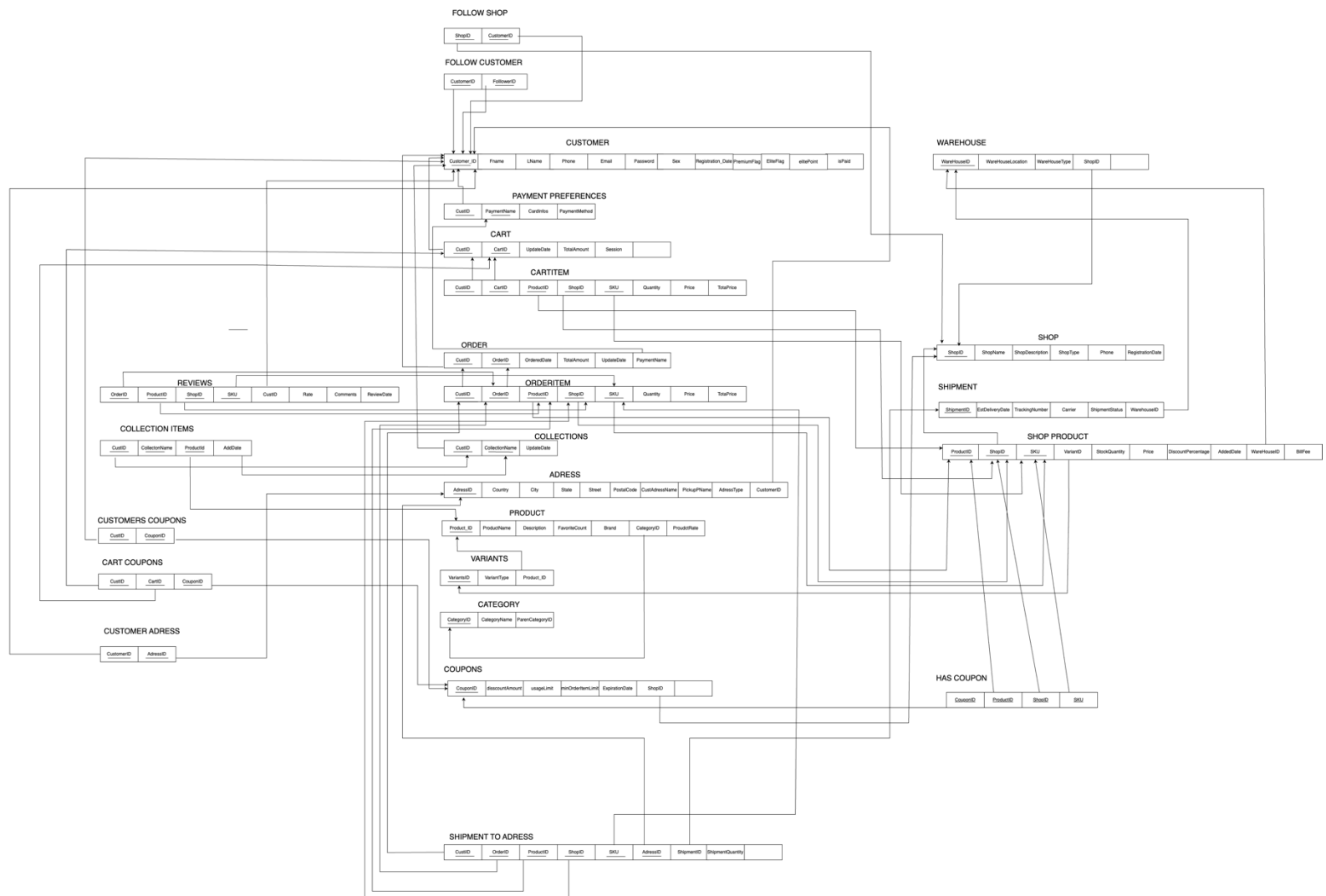
Step 6

- -----

Step 7

- SHIPMENTORGANIZATION (CUSTOMER.CustomerID, ORDER.OrderID, PRODUCT.ProductID, SHOP.ShopId, SHOPPRODUCT.SKU, SHIPMENT.ShipmentID, ADDRESS.AddressID, ShipmentQuantity)

RELATIONAL DATABASE SCHEMA



```

-- Create base tables
CREATE TABLE CUSTOMER (
    CustomerID INT PRIMARY KEY,
    Phone VARCHAR(20),
    Email VARCHAR(255) NOT NULL UNIQUE,
    Password VARCHAR(255) NOT NULL,
    Fname VARCHAR(50),
    Lname VARCHAR(50),
    RegistrationDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    Sex CHAR(1),
    EliteFlag BOOLEAN DEFAULT FALSE,
    PremiumFlag BOOLEAN DEFAULT FALSE,
    ElitePoint INT DEFAULT 0,
    IsPaid BOOLEAN DEFAULT FALSE
);

CREATE TABLE SHOP (
    ShopID INT PRIMARY KEY,
    Phone VARCHAR(20),
    Description TEXT,
    RegistrationDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ShopName VARCHAR(100) NOT NULL,
    ShopType VARCHAR(50)
);

CREATE TABLE CATEGORY (
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(100) NOT NULL,
    ParentCategoryID INT,
    FOREIGN KEY (ParentCategoryID) REFERENCES CATEGORY(CategoryID)
);

CREATE TABLE PRODUCT (
    ProductID INT PRIMARY KEY,
    Favorite_Count INT DEFAULT 0,
    ProductName VARCHAR(255) NOT NULL,
    Brand VARCHAR(100),
    ProductRate DECIMAL(3,2) DEFAULT 0,
    Description TEXT,
    CategoryID INT,
    FOREIGN KEY (CategoryID) REFERENCES CATEGORY(CategoryID)
);

CREATE TABLE VARIANTS (
    VariantID INT PRIMARY KEY,
    VariantType VARCHAR(50) NOT NULL,
    ProductID INT,
    FOREIGN KEY (ProductID) REFERENCES PRODUCT(ProductID)
);

CREATE TABLE WAREHOUSE (
    WarehouseID INT PRIMARY KEY,
    Location VARCHAR(255) NOT NULL,
    WarehouseType VARCHAR(50),

```

```

        ShopID INT,
        FOREIGN KEY (ShopID) REFERENCES SHOP(ShopID)
    );

CREATE TABLE ADDRESS (
    AddressID INT PRIMARY KEY,
    Country VARCHAR(100) NOT NULL,
    State VARCHAR(100),
    City VARCHAR(100) NOT NULL,
    Street VARCHAR(255),
    Postal_Code VARCHAR(20),
    AddressType VARCHAR(50),
    CustAddresName VARCHAR(100) UNIQUE,
    PickUPPointName VARCHAR(100) UNIQUE,
    CustomerID INT,
    FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID)
);

CREATE TABLE SHIPMENT (
    ShipmentID INT PRIMARY KEY,
    Est_Delivery_Date TIMESTAMP,
    TrackingNumber VARCHAR(100) UNIQUE,
    ShipStatus VARCHAR(50),
    Carrier VARCHAR(100),
    WarehouseID INT,
    FOREIGN KEY (WarehouseID) REFERENCES WAREHOUSE(WarehouseID)
);

CREATE TABLE COUPON (
    CouponID INT PRIMARY KEY,
    DiscountAmount DECIMAL(10,2) NOT NULL,
    MinOrderLimit DECIMAL(10,2),
    UsageLimit INT,
    ExpirationDate TIMESTAMP,
    ShopID INT,
    FOREIGN KEY (ShopID) REFERENCES SHOP(ShopID)
);

-- Create tables with composite keys and relationships
CREATE TABLE SHOPPRODUCT (
    SKU VARCHAR(50),
    ProductID INT,
    ShopID INT,
    Price DECIMAL(10,2) NOT NULL,
    Stock INT DEFAULT 0,
    ImportFee DECIMAL(10,2),
    DiscountPercentage DECIMAL(5,2),
    AddedDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    VariantID INT,
    WarehouseID INT,
    PRIMARY KEY (SKU, ProductID, ShopID),
    FOREIGN KEY (ProductID) REFERENCES PRODUCT(ProductID),
    FOREIGN KEY (ShopID) REFERENCES SHOP(ShopID),
    FOREIGN KEY (VariantID) REFERENCES VARIANTS(VariantID),

```



```

        FOREIGN KEY (WarehouseID) REFERENCES Warehouse(WarehouseID)

);

CREATE TABLE PAYMENTPREFERENCES (
    CustomerID INT,
    PaymentName VARCHAR(50),
    PaymentType VARCHAR(100),
    CardInformations TEXT,
    PRIMARY KEY (CustomerID, PaymentName),
    FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID)
);

CREATE TABLE CART (
    CustomerID INT,
    CartID INT,
    TotalAmount DECIMAL(10,2),
    CartSession VARCHAR(255),
    UpdateDate TIMESTAMP,
    PRIMARY KEY (CustomerID, CartID),
    FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID)
);

CREATE TABLE COLLECTIONS (
    CustomerID INT,
    CollectionName VARCHAR(100),
    UpdateDate TIMESTAMP,
    PRIMARY KEY (CustomerID, CollectionName),
    FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID)
);

CREATE TABLE "ORDER" (
    CustomerID INT,
    OrderID INT,
    TotalAmount DECIMAL(10,2),
    OrderedDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UpdateDate TIMESTAMP,
    PaymentName VARCHAR(50),
    PRIMARY KEY (CustomerID, OrderID),
    FOREIGN KEY (CustomerID, PaymentName) REFERENCES
PAYMENTPREFERENCES(CustomerID, PaymentName)
);

-- Create relationship tables
CREATE TABLE FOLLOWSHOPS (
    CustomerID INT,
    ShopID INT,
    PRIMARY KEY (CustomerID, ShopID),
    FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID),
    FOREIGN KEY (ShopID) REFERENCES SHOP(ShopID)
);

CREATE TABLE CUSTOMERCOUPON (
    CustomerID INT,

```

```

        CouponID INT,
        PRIMARY KEY (CustomerID, CouponID),
        FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID),
        FOREIGN KEY (CouponID) REFERENCES COUPON(CouponID)
    );

CREATE TABLE FOLLOWCUSTOMER (
    CustomerID INT,
    FollowedCustomerID INT,
    PRIMARY KEY (CustomerID, FollowedCustomerID),
    FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID),
    FOREIGN KEY (FollowedCustomerID) REFERENCES CUSTOMER(CustomerID)
);

CREATE TABLE HAS_COUPON (
    CouponID INT,
    SKU VARCHAR(50),
    ProductID INT,
    ShopID INT,
    PRIMARY KEY (CouponID, SKU, ProductID, ShopID),
    FOREIGN KEY (CouponID) REFERENCES COUPON(CouponID),
    FOREIGN KEY (SKU, ProductID, ShopID) REFERENCES SHOPPRODUCT(SKU,
ProductID, ShopID)
);

CREATE TABLE CARTCOUPON (
    CouponID INT,
    CustomerID INT,
    CartID INT,
    PRIMARY KEY (CouponID, CustomerID, CartID),
    FOREIGN KEY (CouponID) REFERENCES COUPON(CouponID),
    FOREIGN KEY (CustomerID, CartID) REFERENCES CART(CustomerID, CartID)
);

CREATE TABLE COLLECTIONITEMS (
    CollectionName VARCHAR(100),
    CustomerID INT,
    ProductID INT,
    PRIMARY KEY (CollectionName, CustomerID, ProductID),
    FOREIGN KEY (CustomerID, CollectionName) REFERENCES
COLLECTIONS(CustomerID, CollectionName),
    FOREIGN KEY (ProductID) REFERENCES PRODUCT(ProductID)
);

CREATE TABLE ORDERITEM (
    CustomerID INT,
    OrderID INT,
    ProductID INT,
    ShopID INT,
    SKU VARCHAR(50),
    Quantity INT NOT NULL,
    Price DECIMAL(10,2) NOT NULL,
    Status VARCHAR(50),
    PRIMARY KEY (CustomerID, OrderID, ProductID, ShopID, SKU),

```

```

        FOREIGN KEY (CustomerID, OrderID) REFERENCES "ORDER"(CustomerID,
OrderID),
        FOREIGN KEY (SKU, ProductID, ShopID) REFERENCES SHOPPRODUCT(SKU,
ProductID, ShopID)
);

CREATE TABLE CARTITEM (
    CustomerID INT,
    CartID INT,
    ProductID INT,
    ShopID INT,
    SKU VARCHAR(50),
    Quantity INT NOT NULL,
    Price DECIMAL(10,2) NOT NULL,
    Status VARCHAR(50),
    PRIMARY KEY (CustomerID, CartID, ProductID, ShopID, SKU),
    FOREIGN KEY (CustomerID, CartID) REFERENCES CART(CustomerID, CartID),
    FOREIGN KEY (SKU, ProductID, ShopID) REFERENCES SHOPPRODUCT(SKU,
ProductID, ShopID)
);

CREATE TABLE REVIEWS (
    CustomerID INT,
    OrderID INT,
    ProductID INT,
    ShopID INT,
    SKU VARCHAR(50),
    Rate INT CHECK (Rate BETWEEN 1 AND 5),
    Review_Comment TEXT,
    Review_Date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (CustomerID, OrderID, ProductID, ShopID, SKU),
    FOREIGN KEY (CustomerID, OrderID, ProductID, ShopID, SKU) REFERENCES
ORDERITEM(CustomerID, OrderID, ProductID, ShopID, SKU)
);

CREATE TABLE SHIPMENTORGANIZATION (
    CustomerID INT,
    OrderID INT,
    ProductID INT,
    ShopID INT,
    SKU VARCHAR(50),
    ShipmentID INT,
    AddressID INT,
    ShipmentQuantity INT NOT NULL,
    PRIMARY KEY (CustomerID, OrderID, ProductID, ShopID, SKU, ShipmentID),
    FOREIGN KEY (CustomerID, OrderID, ProductID, ShopID, SKU) REFERENCES
ORDERITEM(CustomerID, OrderID, ProductID, ShopID, SKU),
    FOREIGN KEY (ShipmentID) REFERENCES SHIPMENT(ShipmentID),
    FOREIGN KEY (AddressID) REFERENCES ADDRESS(AddressID)
);

```

```

-- Populate CATEGORY table with hierarchical structure
INSERT INTO CATEGORY (CategoryID, CategoryName, ParentCategoryID) VALUES
(1, 'Electronics', NULL),
(2, 'Smartphones', 1),
(3, 'Laptops', 1),
(4, 'Fashion', NULL),
(5, 'Men''s Fashion', 4),
(6, 'Women''s Fashion', 4),
(7, 'Home & Living', NULL),
(8, 'Furniture', 7),
(9, 'Kitchen', 7);

-- Populate CUSTOMER table with Turkish customers
INSERT INTO CUSTOMER (CustomerID, Phone, Email, Password, Fname, Lname,
Sex, EliteFlag, PremiumFlag, ElitePoint, IsPaid) VALUES
(1, '+90555-0101', 'ali.yilmaz@email.com', 'hash123', 'Ali', 'Yilmaz',
'M', true, true, 1500, true),
(2, '+90555-0102', 'ayse.demir@email.com', 'hash456', 'Ayse', 'Demir',
'F', false, true, 0, true),
(3, '+90555-0103', 'mehmet.kaya@email.com', 'hash789', 'Mehmet', 'Kaya',
'M', false, false, 0, false),
(4, '+90555-0104', 'zeynep.celik@email.com', 'hash101', 'Zeynep', 'Celik',
'F', true, false, 0, true),
(5, '+90555-0105', 'can.ozturk@email.com', 'hash102', 'Can', 'Ozturk',
'M', false, true, 600, true);

-- Populate SHOP table with shops in different Turkish cities
INSERT INTO SHOP (ShopID, Phone, Description, ShopName, ShopType) VALUES
(1, '+90216-1001', 'Premium Electronics Store in Istanbul', 'TechMaster
Istanbul', 'Electronics'),
(2, '+90312-1002', 'Fashion Boutique in Ankara', 'Ankara Style Hub',
'Fashion'),
(3, '+90232-1003', 'Home Goods Store in Izmir', 'Izmir Home Center', 'Home
& Living'),
(4, '+90242-1004', 'Electronics Store in Antalya', 'Antalya Digital',
'Electronics'),
(5, '+90332-1005', 'Fashion Store in Konya', 'Konya Fashion House',
'Fashion');

-- Populate PRODUCT table
INSERT INTO PRODUCT (ProductID, Favorite_Count, ProductName, Brand,
Description, CategoryID) VALUES
(1, 150, 'iPhone 14 Pro', 'Apple', 'Latest iPhone model', 2),
(2, 120, 'Galaxy S23', 'Samsung', 'Latest Samsung flagship', 2),
(3, 200, 'MacBook Pro M2', 'Apple', 'Professional laptop', 3),
(4, 80, 'ThinkPad X1', 'Lenovo', 'Business laptop', 3),
(5, 90, 'Leather Jacket', 'TurkStyle', 'Genuine leather jacket', 5),
(6, 100, 'Evening Dress', 'Boutique', 'Elegant evening dress', 6),
(7, 70, 'Modern Sofa', 'TurkFurniture', 'Contemporary design sofa', 8),
(8, 60, 'Kitchen Set', 'HomeStyle', 'Complete kitchen set', 9);

-- Populate VARIANTS table

```

```

INSERT INTO VARIANTS (VariantID, VariantType, ProductID) VALUES
(1, '128GB Space Gray', 1),
(2, '256GB Space Gray', 1),
(3, '256GB Black', 2),
(4, '512GB Black', 2),
(5, '14" Space Gray', 3),
(6, '16" Space Gray', 3),
(7, 'Black M', 5),
(8, 'Black L', 5),
(9, 'Red S', 6),
(10, 'Red M', 6);

-- Populate WAREHOUSE table with locations in Turkish cities
INSERT INTO WAREHOUSE (WarehouseID, Location, WarehouseType, ShopID)
VALUES
(1, 'Istanbul European Side', 'FBA', null),
(2, 'Ankara Central', 'FBA', null),
(3, 'Izmir Port Area', 'FBA', null),
(4, 'Istanbul Asian Side', 'FBA', null),
(5, 'Antalya Coast', 'FBM', 4);

-- Populate ADDRESS table with Turkish addresses
INSERT INTO ADDRESS (AddressID, Country, State, City, Street, Postal_Code,
AddressType, CustAddressName, CustomerID) VALUES
(1, 'Turkey', 'Marmara', 'Istanbul', 'Bagdat Street', '34710',
'CustomerAddress', 'Home Address 1', 1),
(2, 'Turkey', 'Central Anatolia', 'Ankara', 'Tunali Street', '06550',
'CustomerAddress', 'Home Address 2', 2),
(3, 'Turkey', 'Aegean', 'Izmir', 'Alsancak Street', '35220',
'CustomerAddress', 'Work Address 1', 3),
(4, 'Turkey', 'Mediterranean', 'Antalya', 'Konyaalti Street', '07070',
'CustomerAddress', 'Home Address 3', 4),
(5, 'Turkey', 'Central Anatolia', 'Konya', 'Meram Street', '42090',
'CustomerAddress', 'MyHome', 5),
(6, 'Turkey', 'Central Anatolia', 'Konya', 'Mevlana Street', '42200',
'CustomerAddress', 'Brother Home', 5),
(7, 'Turkey', 'BlackSea', 'Trabzon', 'Macka Street', '61030',
'PickupPointAdress', 'Burak Petshop', null);

-- Populate SHIPMENT table
INSERT INTO SHIPMENT (ShipmentID, Est_Delivery_Date, TrackingNumber,
ShipStatus, Carrier, WarehouseID) VALUES
(1, CURRENT_TIMESTAMP + INTERVAL '3 days', 'TR100001', 'In Transit',
'Turkish Cargo', 1),
(2, CURRENT_TIMESTAMP + INTERVAL '2 days', 'TR100002', 'Processing', 'Aras
Cargo', 2),
(3, CURRENT_TIMESTAMP + INTERVAL '4 days', 'TR100003', 'Delivered', 'MNG
Cargo', 3),
(4, CURRENT_TIMESTAMP + INTERVAL '1 day', 'TR100004', 'In Transit', 'PTT
Cargo', 4),
(5, CURRENT_TIMESTAMP + INTERVAL '5 days', 'TR100005', 'Processing',
'Yurtici Cargo', 5),
(6, CURRENT_TIMESTAMP + INTERVAL '2 days', 'TR100006', 'In Transit',
'Turkish Cargo', 1),

```

```

(7, CURRENT_TIMESTAMP + INTERVAL '3 days', 'TR100007', 'Processing', 'Aras
Cargo', 2),
(8, CURRENT_TIMESTAMP + INTERVAL '1 day', 'TR100008', 'In Transit', 'MNG
Cargo', 3),
(9, CURRENT_TIMESTAMP + INTERVAL '4 days', 'TR100009', 'Processing', 'PTT
Cargo', 4),
(10, CURRENT_TIMESTAMP + INTERVAL '2 days', 'TR100010', 'In Transit',
'Aras Cargo', 5),
(11, CURRENT_TIMESTAMP + INTERVAL '3 days', 'TR100011', 'In Transit',
'Yurtici Cargo', 5),
(12, CURRENT_TIMESTAMP + INTERVAL '2 days', 'TR100012', 'In Transit',
'Yurtici Cargo', 5);

```

```
-- Populate COUPON table
```

```

INSERT INTO COUPON (CouponID, DiscountAmount, MinOrderLimit, UsageLimit,
ExpirationDate, ShopID) VALUES
(1, 100.00, 1000.00, 100, CURRENT_TIMESTAMP + INTERVAL '30 days', 1),
(2, 50.00, 500.00, 200, CURRENT_TIMESTAMP + INTERVAL '15 days', 2),
(3, 200.00, 2000.00, 50, CURRENT_TIMESTAMP + INTERVAL '45 days', 3),
(4, 75.00, 750.00, 150, CURRENT_TIMESTAMP + INTERVAL '60 days', 4),
(5, 150.00, 1500.00, 75, CURRENT_TIMESTAMP + INTERVAL '20 days', 5),
(6, 250.00, 2500.00, 1000, CURRENT_TIMESTAMP + INTERVAL '30 days', null),
-- Platform-wide discount for high-value orders
(7, 100.00, 1000.00, 2000, CURRENT_TIMESTAMP + INTERVAL '45 days', null),
-- Mid-range platform discount
(8, 50.00, 500.00, 5000, CURRENT_TIMESTAMP + INTERVAL '60 days', null);
-- Entry-level platform discount

```

```
-- Now add HAS_COUPON entries for platform-wide coupons
```

```
-- Populate SHOPPRODUCT table with prices in Turkish Lira
```

```

INSERT INTO SHOPPRODUCT (SKU, ProductID, ShopID, Price, Stock, ImportFee,
DiscountPercentage, VariantID, WarehouseID) VALUES
('IP14-128-SG', 1, 1, 45000.00, 50, 1000.00, 5.00, 1, 1),
('IP14-256-SG', 1, 1, 50000.00, 30, 1000.00, 5.00, 2, 4),
('GS23-256-B', 2, 4, 35000.00, 40, 800.00, 10.00, 3, 5),
('GS23-512-B', 2, 4, 40000.00, 25, 800.00, 10.00, 4, 5),
('MBP-14-SG', 3, 1, 55000.00, 20, 1200.00, 0.00, 5, 1),
('MBP-16-SG', 3, 1, 65000.00, 15, 1200.00, 0.00, 6, 1),
('LJ-BLACK-M', 5, 2, 3000.00, 100, 100.00, 15.00, 7, 2),
('LJ-BLACK-L', 5, 2, 3000.00, 80, 100.00, 15.00, 8, 2),
('ED-RED-S', 6, 2, 2500.00, 60, 80.00, 20.00, 9, 2),
('ED-RED-M', 6, 2, 2500.00, 45, 80.00, 20.00, 10, 2),
('IP14-128-SG-2', 1, 2, 46000.00, 30, 1000.00, 3.00, 1, 2),
('IP14-128-SG-3', 1, 3, 45500.00, 25, 1000.00, 4.00, 1, 3),
('GS23-256-B-1', 2, 1, 34500.00, 35, 800.00, 8.00, 3, 4),
('GS23-256-B-3', 2, 3, 35500.00, 30, 800.00, 7.00, 3, 3),
('MBP-14-SG-4', 3, 4, 56000.00, 15, 1200.00, 2.00, 5, 4),
('TKS-12-NM-1', 8, 4, 1430, 3, 0, 0, null, 4);

```

```

INSERT INTO PAYMENTPREFERENCES (CustomerID, PaymentName, PaymentType,
CardInformations)
VALUES

```

```
-- Customer 1 (Ali Yilmaz) payment methods
```

```

        (1, 'Ziraat Credit Card', 'CARD', 'Card Type: Credit, Bank: Ziraat,
Last4: 1234, Expiry: 12/25'),
        (1, 'Garanti Debit', 'CARD', 'Card Type: Debit, Bank: Garanti, Last4:
5678, Expiry: 03/26'),
        (1, 'YapiKredi Contactless', 'CARD', 'Card Type: Credit, Bank:
YapiKredi, Last4: 9012, Expiry: 08/24'),

        -- Customer 2 (Ayse Demir) payment methods
        (2, 'IsBank Credit', 'CARD', 'Card Type: Credit, Bank: Is Bankasi,
Last4: 3456, Expiry: 05/25'),
        (2, 'Akbank World', 'CARD', 'Card Type: Credit, Bank: Akbank, Last4:
7890, Expiry: 11/24'),

        -- Customer 3 (Mehmet Kaya) payment methods
        (3, 'Halkbank Paraf', 'CARD', 'Card Type: Credit, Bank: Halkbank,
Last4: 2345, Expiry: 07/26'),
        (3, 'Vakifbank Debit', 'CARD', 'Card Type: Debit, Bank: Vakifbank,
Last4: 6789, Expiry: 09/25'),

        -- Customer 4 (Zeynep Celik) payment methods
        (4, 'Denizbank Credit', 'CARD', 'Card Type: Credit, Bank: Denizbank,
Last4: 4567, Expiry: 04/25'),
        (4, 'QNB Card', 'CARD', 'Card Type: Credit, Bank: QNB Finansbank,
Last4: 8901, Expiry: 10/24'),
        (4, 'TEB Debit', 'CARD', 'Card Type: Debit, Bank: TEB, Last4: 3456,
Expiry: 06/26'),

        -- Customer 5 (Can Ozturk) payment methods
        (5, 'ING Bank Credit', 'CARD', 'Card Type: Credit, Bank: ING Bank,
Last4: 5678, Expiry: 02/25'),
        (5, 'Garanti Bonus', 'CARD', 'Card Type: Credit, Bank: Garanti, Last4:
9012, Expiry: 12/24');

-- Populate CART table
INSERT INTO CART (CustomerID, CartID, TotalAmount, CartSession,
UpdateDate) VALUES
(1, 1, 91000.00, 'SESSION1', CURRENT_TIMESTAMP - INTERVAL '2 hours'),
(2, 1, 35000.00, 'SESSION2', CURRENT_TIMESTAMP - INTERVAL '23 hours'),
(3, 1, 3000.00, 'SESSION3', CURRENT_TIMESTAMP - INTERVAL '45 hours'),
(4, 1, 2500.00, 'SESSION4', CURRENT_TIMESTAMP - INTERVAL '72 hours'),
(5, 1, 55000.00, 'SESSION5', CURRENT_TIMESTAMP - INTERVAL '89 hours');

-- Populate ORDER table
INSERT INTO "ORDER" (CustomerID, OrderID, TotalAmount, OrderedDate,
UpdateDate, PaymentName) VALUES
(1, 1, 91000.00, CURRENT_TIMESTAMP - INTERVAL '5 days',
CURRENT_TIMESTAMP, 'Ziraat Credit Card'),
(2, 1, 35000.00, CURRENT_TIMESTAMP - INTERVAL '4 days',
CURRENT_TIMESTAMP, 'IsBank Credit'),
(3, 1, 3000.00, CURRENT_TIMESTAMP - INTERVAL '3 days',
CURRENT_TIMESTAMP, 'Halkbank Paraf'),
(4, 1, 2500.00, CURRENT_TIMESTAMP - INTERVAL '2 days',
CURRENT_TIMESTAMP, 'Denizbank Credit'),

```

```

(1, 2, 70000.00, CURRENT_TIMESTAMP - INTERVAL '2 hours',
CURRENT_TIMESTAMP, 'Garanti Debit'),
(1, 3, 56000.00, CURRENT_TIMESTAMP - INTERVAL '4 hours',
CURRENT_TIMESTAMP, 'Yapikredi Contactless'),
(1, 4, 45500.00, CURRENT_TIMESTAMP - INTERVAL '6 hours',
CURRENT_TIMESTAMP, 'Ziraat Credit Card'),
(2, 2, 46000.00, CURRENT_TIMESTAMP - INTERVAL '3 hours',
CURRENT_TIMESTAMP, 'Akbank World'),
(2, 3, 35500.00, CURRENT_TIMESTAMP - INTERVAL '5 hours',
CURRENT_TIMESTAMP, 'IsBank Credit'),
(5, 1, 61000, CURRENT_TIMESTAMP - INTERVAL '3 days',
CURRENT_TIMESTAMP, 'ING Bank Credit');

```

-- Populate ORDERITEM table

```

INSERT INTO ORDERITEM (CustomerID, OrderID, ProductID, ShopID, SKU,
Quantity, Price, Status) VALUES
(1, 1, 1, 1, 'IP14-128-SG', 1, 45000.00, 'Delivered'),
(2, 1, 2, 4, 'GS23-256-B', 1, 35000.00, 'In Transit'),
(3, 1, 5, 2, 'LJ-BLACK-M', 1, 3000.00, 'Processing'),
(4, 1, 6, 2, 'ED-RED-S', 1, 2500.00, 'Shipped'),
(5, 1, 3, 1, 'MBP-14-SG', 1, 55000.00, 'Processing'),
(1, 1, 1, 2, 'IP14-128-SG-2', 1, 46000.00, 'Processing'),
(2, 1, 2, 1, 'GS23-256-B-1', 1, 34500.00, 'Delivered'),
(3, 1, 1, 3, 'IP14-128-SG-3', 1, 45500.00, 'In Transit'),
(5, 1, 5, 2, 'LJ-BLACK-L', 2, 3000.00, 'In Transit'),
(1, 2, 2, 1, 'GS23-256-B-1', 2, 34500.00, 'Processing'),
(1, 3, 3, 4, 'MBP-14-SG-4', 1, 56000.00, 'Processing'),
(1, 4, 1, 3, 'IP14-128-SG-3', 1, 45500.00, 'Processing'),
(2, 2, 1, 2, 'IP14-128-SG-2', 1, 46000.00, 'Processing'),
(2, 3, 2, 3, 'GS23-256-B-3', 1, 35500.00, 'Processing');

```

```

INSERT INTO CARTITEM (CustomerID, CartID, ProductID, ShopID, SKU,
Quantity, Price, Status) VALUES

```

-- Cart for Customer 1 (Multiple items)

```

(1, 1, 1, 1, 'IP14-128-SG', 1, 45000.00, 'Active'),
(1, 1, 2, 4, 'GS23-256-B', 1, 35000.00, 'Active'),

```

-- Cart for Customer 2 (Multiple electronics)

```

(2, 1, 3, 1, 'MBP-14-SG', 1, 55000.00, 'Active'),
(2, 1, 2, 1, 'GS23-256-B-1', 1, 34500.00, 'Active'),

```

-- Cart for Customer 3 (Fashion items)

```

(3, 1, 5, 2, 'LJ-BLACK-M', 2, 3000.00, 'Active'),
(3, 1, 6, 2, 'ED-RED-S', 1, 2500.00, 'Active'),

```

-- Cart for Customer 4 (Mixed items)

```

(4, 1, 6, 2, 'ED-RED-M', 1, 2500.00, 'Active'),
(4, 1, 1, 3, 'IP14-128-SG-3', 1, 45500.00, 'Active'),

```

-- Cart for Customer 5 (High-value electronics)

```

(5, 1, 3, 1, 'MBP-14-SG', 1, 55000.00, 'Active'),
(5, 1, 1, 1, 'IP14-256-SG', 1, 50000.00, 'Active');

```



```

-- Populate REVIEWS table
INSERT INTO REVIEWS (CustomerID, OrderID, ProductID, ShopID, SKU, Rate,
Review_Comment) VALUES
(1, 1, 1, 1, 'IP14-128-SG', 5, 'Excellent product and fast delivery!'),
(2, 1, 2, 4, 'GS23-256-B', 4, 'Good phone, slightly expensive'),
(3, 1, 5, 2, 'LJ-BLACK-M', 5, 'Perfect fit and great quality'),
(4, 1, 6, 2, 'ED-RED-S', 4, 'Beautiful dress, fast shipping'),
(5, 1, 3, 1, 'MBP-14-SG', 5, 'Amazing laptop, worth every penny'),
(1, 1, 1, 2, 'IP14-128-SG-2', 4, 'Good service and product'),
(2, 1, 2, 1, 'GS23-256-B-1', 5, 'Excellent phone and quick delivery'),
(3, 1, 1, 3, 'IP14-128-SG-3', 4, 'Satisfied with the purchase');

-- Populate relationship tables
INSERT INTO FOLLOWSHOPS (CustomerID, ShopID) VALUES
(1, 1), (1, 2), (2, 2), (3, 3), (4, 4);

INSERT INTO CUSTOMERCOUPON (CustomerID, CouponID) VALUES
(1, 1), (2, 2), (3, 3), (4, 4), (5, 5);

INSERT INTO CARTCOUPON (CouponID, CustomerID, CartID)
VALUES
-- Customer 1's cart (total amount 91000 TL) can use high-value
coupons
(1, 1, 1), -- 100 TL off from TechMaster
(3, 1, 1), -- 200 TL off from Izmir Home Center

-- Customer 2's cart (total amount 35000 TL)
(2, 2, 1), -- 50 TL off from Ankara Style Hub

-- Customer 5's cart (total amount 55000 TL)
(4, 5, 1); -- 75 TL off from Antalya Digital
INSERT INTO HAS_COUPON (CouponID, SKU, ProductID, ShopID)
VALUES
-- TechMaster Istanbul (ShopID 1) coupons
(1, 'IP14-128-SG', 1, 1), -- iPhone 14 Pro 128GB
(1, 'MBP-14-SG', 3, 1), -- MacBook Pro 14"
(1, 'MBP-16-SG', 3, 1), -- MacBook Pro 16"

-- Ankara Style Hub (ShopID 2) coupons
(2, 'LJ-BLACK-M', 5, 2), -- Leather Jacket M
(2, 'LJ-BLACK-L', 5, 2), -- Leather Jacket L
(2, 'ED-RED-S', 6, 2), -- Evening Dress S
(2, 'ED-RED-M', 6, 2), -- Evening Dress M

-- Antalya Digital (ShopID 4) coupons
(4, 'GS23-256-B', 2, 4), -- Galaxy S23 256GB
(4, 'GS23-512-B', 2, 4), -- Galaxy S23 512GB
(4, 'MBP-14-SG-4', 3, 4); -- MacBook Pro at Antalya Digital

```

```
-- High-value coupon (CouponID 6) for expensive electronics across shops
```

```
(6, 'IP14-128-SG', 1, 1),      -- iPhone from TechMaster
(6, 'IP14-128-SG-2', 1, 2),    -- iPhone from Ankara Style
(6, 'IP14-128-SG-3', 1, 3),    -- iPhone from Izmir Home
(6, 'MBP-14-SG', 3, 1),        -- MacBook from TechMaster
(6, 'MBP-14-SG-4', 3, 4),      -- MacBook from Antalya Digital
```

```
-- Mid-range coupon (CouponID 7) for mid-priced items
```

```
(7, 'GS23-256-B', 2, 4),      -- Galaxy from Antalya Digital
(7, 'GS23-256-B-1', 2, 1),    -- Galaxy from TechMaster
(7, 'GS23-256-B-3', 2, 3),    -- Galaxy from Izmir Home
```

```
-- Entry-level coupon (CouponID 8) for fashion items
```

```
(8, 'LJ-BLACK-M', 5, 2),      -- Leather Jacket from Ankara Style
(8, 'ED-RED-S', 6, 2),        -- Evening Dress from Ankara Style
(8, 'ED-RED-M', 6, 2);        -- Evening Dress from Ankara Style
```

```
INSERT INTO FOLLOWCUSTOMER (CustomerID, FollowedCustomerID) VALUES
(1, 2), (2, 1), (3, 1), (4, 1), (5, 1);
```

```
-- Populate SHIPMENTORGANIZATION table
```

```
INSERT INTO SHIPMENTORGANIZATION (CustomerID, OrderID, ProductID, ShopID,
SKU, ShipmentID, AddressID, ShipmentQuantity) VALUES
```

```
(1, 1, 1, 1, 'IP14-128-SG', 1, 1, 1),
(2, 1, 2, 4, 'GS23-256-B', 2, 2, 1),
(3, 1, 5, 2, 'LJ-BLACK-M', 3, 3, 1),
(4, 1, 6, 2, 'ED-RED-S', 4, 4, 1),
(5, 1, 3, 1, 'MBP-14-SG', 5, 5, 1),
(5, 1, 5, 2, 'LJ-BLACK-L', 11, 5, 1),
(5, 1, 5, 2, 'LJ-BLACK-L', 12, 6, 1),
(1, 2, 2, 1, 'GS23-256-B-1', 6, 1, 1),
(1, 3, 3, 4, 'MBP-14-SG-4', 7, 1, 1),
(1, 4, 1, 3, 'IP14-128-SG-3', 8, 1, 1),
(2, 2, 1, 2, 'IP14-128-SG-2', 9, 2, 1),
(2, 3, 2, 3, 'GS23-256-B-3', 10, 2, 1);
```

```
INSERT INTO COLLECTIONS (CustomerID, CollectionName, UpdateDate) VALUES
```

```
-- Customer 1's collections
```

```
(1, 'My Electronics Wishlist', CURRENT_TIMESTAMP),
(1, 'Gift Ideas', CURRENT_TIMESTAMP - INTERVAL '5 days'),
(1, 'favorites', CURRENT_TIMESTAMP - INTERVAL '10 days'),
```

```
-- Customer 2's collections
```

```
(2, 'Fashion Favorites', CURRENT_TIMESTAMP),
(2, 'Tech Gadgets', CURRENT_TIMESTAMP - INTERVAL '3 days'),
(2, 'favorites', CURRENT_TIMESTAMP - INTERVAL '7 days');
```

```
INSERT INTO COLLECTIONITEMS (CollectionName, CustomerID, ProductID) VALUES
-- Customer 1's collection items
('My Electronics Wishlist', 1, 1), -- iPhone
('My Electronics Wishlist', 1, 2), -- Galaxy
('My Electronics Wishlist', 1, 3), -- MacBook
('Gift Ideas', 1, 5),             -- Leather Jacket
('Gift Ideas', 1, 6),             -- Evening Dress
('favorites', 1, 4),              -- ThinkPad

-- Customer 2's collection items
('Fashion Favorites', 2, 5),      -- Leather Jacket
('Fashion Favorites', 2, 6),      -- Evening Dress
('Tech Gadgets', 2, 1),          -- iPhone
('Tech Gadgets', 2, 2),          -- Galaxy
('favorites', 2, 3);             -- MacBook
```

```

-- Add check constraints to existing tables
ALTER TABLE SHOPPRODUCT
ADD CONSTRAINT check_price_positive
CHECK (Price > 0);

ALTER TABLE COUPON
ADD CONSTRAINT check_valid_discount
CHECK (DiscountAmount > 0 AND DiscountAmount <= MinOrderLimit);
-- Trigger 1: Update product rating when a new review is added
CREATE OR REPLACE FUNCTION update_product_rating()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE PRODUCT
    SET ProductRate = (
        SELECT AVG(Rate)
        FROM REVIEWS
        WHERE ProductID = NEW.ProductID
    )
    WHERE ProductID = NEW.ProductID;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER after_review_insert
AFTER INSERT ON REVIEWS
FOR EACH ROW
EXECUTE FUNCTION update_product_rating();

-- Trigger 2: Update cart total amount when items are added/modified
CREATE OR REPLACE FUNCTION update_cart_total()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE CART
    SET TotalAmount = (
        SELECT SUM(Price * Quantity)
        FROM CARTITEM
        WHERE CustomerID = NEW.CustomerID AND CartID = NEW.CartID
    ),
    UpdateDate = CURRENT_TIMESTAMP
    WHERE CustomerID = NEW.CustomerID AND CartID = NEW.CartID;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER after_cartitem_change
AFTER INSERT OR UPDATE OR DELETE ON CARTITEM
FOR EACH ROW
EXECUTE FUNCTION update_cart_total();

-- Trigger 3: Update stock when order is placed
CREATE OR REPLACE FUNCTION update_stock_after_order()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE SHOPPRODUCT

```

```

        SET Stock = Stock - NEW.Quantity
        WHERE SKU = NEW.SKU AND ProductID = NEW.ProductID AND ShopID =
NEW.ShopID;
        RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER after_order_insert
AFTER INSERT ON ORDERITEM
FOR EACH ROW
EXECUTE FUNCTION update_stock_after_order();

CREATE OR REPLACE FUNCTION update_favorite_count()
RETURNS TRIGGER AS $$
BEGIN
    -- If the collection name is 'favorites', increment the favorite count
    IF NEW.CollectionName = 'favorites' THEN
        UPDATE PRODUCT
        SET Favorite_Count = Favorite_Count + 1
        WHERE ProductID = NEW.ProductID;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER increment_favorite_count
AFTER INSERT ON COLLECTIONITEMS
FOR EACH ROW
EXECUTE FUNCTION update_favorite_count();

-- INSERT statements
INSERT INTO CUSTOMER (CustomerID, Phone, Email, Password, Fname, Lname,
Sex)
VALUES (6, '555-0123', 'john@example.com', 'hashedpass123', 'John', 'Doe',
'M');

INSERT INTO PRODUCT (ProductID, ProductName, Brand,Description,
CategoryID)
VALUES (9,'Gaming Laptop', 'TechBrand','Very fast gaming laptop', 3);

INSERT INTO SHOP (ShopID, Phone, ShopName, ShopType)
VALUES (6, '555-9999', 'Electronics Hub', 'Electronics');

-- UPDATE statements
UPDATE CUSTOMER
SET EliteFlag = TRUE, ElitePoint = 100
WHERE CustomerID = 1;

```

```

UPDATE PRODUCT
SET ProductRate = 4
WHERE ProductID = 1;

UPDATE SHOP
SET Description = 'Best electronics shop in town'
WHERE ShopID = 1;

-- DELETE statements
DELETE FROM CUSTOMER
WHERE CustomerID = 6;

DELETE FROM PRODUCT
WHERE ProductID = 9;

DELETE FROM SHOP
WHERE ShopID =6;

-- Find all products and their categories
SELECT p.ProductName, c.CategoryName
FROM PRODUCT p
JOIN CATEGORY c ON p.CategoryID = c.CategoryID;

-- Find all shops and their follower count
SELECT s.ShopName, COUNT(f.CustomerID) as FollowerCount
FROM SHOP s
LEFT JOIN FOLLOWSHOPS f ON s.ShopID = f.ShopID
GROUP BY s.ShopName;

-- Find reviewed products in each category with their reviews
SELECT c.CategoryName, p.ProductName, r.Review_Comment, COUNT(r.CustomerID)
as ReviewCount, ROUND(AVG(r.Rate)) as AvgRating
FROM CATEGORY c
JOIN PRODUCT p ON c.CategoryID = p.CategoryID
JOIN REVIEWS r ON p.ProductID = r.ProductID
GROUP BY c.CategoryName, p.ProductName, r.Review_Comment;

-- Find customer order history with product and shop details
SELECT c.Fname, c.Lname, o.OrderID, p.ProductName, s.ShopName, oi.Quantity,
oi.Price
FROM CUSTOMER c
JOIN "ORDER" o ON c.CustomerID = o.CustomerID
JOIN ORDERITEM oi ON o.CustomerID = oi.CustomerID AND o.OrderID =
oi.OrderID
JOIN PRODUCT p ON oi.ProductID = p.ProductID
JOIN SHOP s ON oi.ShopID = s.ShopID
ORDER BY c.fname ASC ;

--Collections per customer and products in the collections
SELECT
    c.CustomerID,

```

```

        cu.Fname,
        cu.Lname,
        c.CollectionName,
        COUNT(ci.ProductID) as ItemCount,
        STRING_AGG(p.ProductName, ', ') as Products
FROM COLLECTIONS c
LEFT JOIN COLLECTIONITEMS ci ON c.CustomerID = ci.CustomerID
    AND c.CollectionName = ci.CollectionName
LEFT JOIN PRODUCT p ON ci.ProductID = p.ProductID
JOIN CUSTOMER cu ON cu.CustomerID = c.CustomerID
GROUP BY c.CustomerID, c.CollectionName, cu.Fname,
        cu.Lname
ORDER BY c.CustomerID, c.CollectionName;

```

--Price Comparison Across Shops

```

WITH ProductShopCount AS (
    SELECT ProductID
    FROM SHOPPRODUCT
    GROUP BY ProductID
    HAVING COUNT(DISTINCT ShopID) > 1
)
SELECT
    p.ProductID,
    s.ShopID,
    p.ProductName,
    p.Brand,
    v.VariantType,
    s.ShopName,
    sp.Price as OriginalPrice,
    sp.DiscountPercentage,
    ROUND(sp.Price * (1 - sp.DiscountPercentage/100), 2) as
DiscountedPrice,
    sp.Stock,
    p.ProductRate,
    COUNT(r.CustomerID) as ReviewCount
FROM PRODUCT p
JOIN SHOPPRODUCT sp ON p.ProductID = sp.ProductID
JOIN SHOP s ON sp.ShopID = s.ShopID
JOIN VARIANTS v ON sp.VariantID = v.VariantID
LEFT JOIN REVIEWS r ON p.ProductID = r.ProductID AND s.ShopID = r.ShopID
JOIN ProductShopCount psc ON p.ProductID = psc.ProductID
GROUP BY p.ProductID, p.ProductName, p.Brand, v.VariantType, s.ShopID,
        s.ShopName,
        sp.Price, sp.DiscountPercentage, sp.Stock, p.ProductRate
ORDER BY p.ProductID;

```

--Find trending products based on recent orders, ratings, and favorites:

```

SELECT
    p.ProductID,

```

```

        p.ProductName,
        COUNT(DISTINCT oi.OrderID) as recent_orders,
        ROUND(AVG(r.Rate),2) as avg_rating,
        p.Favorite_Count,
        ROUND((COUNT(DISTINCT oi.OrderID) * 0.50 + COALESCE(AVG(r.Rate), 0) *
0.35 + p.Favorite_Count * 0.15),2) as trend_score
FROM PRODUCT p
LEFT JOIN ORDERITEM oi ON p.ProductID = oi.ProductID
    AND oi.OrderID IN (
        SELECT OrderID
        FROM "ORDER"
        WHERE OrderedDate >= CURRENT_DATE - INTERVAL '30 days'
    )
LEFT JOIN REVIEWS r ON p.ProductID = r.ProductID
GROUP BY p.ProductID, p.ProductName, p.Favorite_Count
HAVING COUNT(DISTINCT oi.OrderID) > 0
ORDER BY trend_score DESC
LIMIT 20;

```

--. Stock Level Tracking Across All Shops

```

SELECT
    p.ProductID,
    p.ProductName,
    p.Brand,
    s.ShopID,
    s.ShopName,
    sp.SKU,
    v.VariantType,
    sp.Stock,
    CASE
        WHEN sp.Stock = 0 THEN 'Out of Stock'
        WHEN sp.Stock < 10 THEN 'Low Stock'
        WHEN sp.Stock < 50 THEN 'Moderate Stock'
        ELSE 'Good Stock'
    END as StockStatus,
    w.Location as WarehouseLocation
FROM PRODUCT p
JOIN SHOPPRODUCT sp ON p.ProductID = sp.ProductID
JOIN SHOP s ON sp.ShopID = s.ShopID
JOIN VARIANTS v ON sp.VariantID = v.VariantID
LEFT JOIN WAREHOUSE w ON sp.WarehouseID = w.WarehouseID
ORDER BY sp.Stock ASC;

```

-- Customer lifetime value analysis with their preferred categories:

```

SELECT
    c.CustomerID,
    c.Email,
    COUNT(DISTINCT o.OrderID) as total_orders,
    SUM(o.TotalAmount) as lifetime_spend,
    STRING_AGG(DISTINCT cat.CategoryName, ', ') as preferred_categories,
    c.ElitePoint,
    CASE
        WHEN c.EliteFlag THEN 'Elite'
    END

```



```

        WHEN c.PremiumFlag THEN 'Premium'
        ELSE 'Regular'
    END as customer_tier
FROM CUSTOMER c
LEFT JOIN "ORDER" o ON c.CustomerID = o.CustomerID
LEFT JOIN ORDERITEM oi ON o.OrderID = oi.OrderID AND o.CustomerID =
oi.CustomerID
LEFT JOIN PRODUCT p ON oi.ProductID = p.ProductID
LEFT JOIN CATEGORY cat ON p.CategoryID = cat.CategoryID
GROUP BY c.CustomerID, c.Email, c.EliteFlag, c.PremiumFlag, c.ElitePoint
ORDER BY lifetime_spend DESC;

```

--Find customers who have abandoned items in their cart

--(useful for follow-up marketing):

```

SELECT
    c.CustomerID,
    c.Email,
    p.ProductName,
    ci.Quantity,
    ci.Price,
    cart.UpdateDate as last_cart_update
FROM CART cart
JOIN CARTITEM ci ON cart.CartID = ci.CartID AND cart.CustomerID =
ci.CustomerID
JOIN CUSTOMER c ON cart.CustomerID = c.CustomerID
JOIN PRODUCT p ON ci.ProductID = p.ProductID
WHERE cart.UpdateDate < CURRENT_TIMESTAMP - INTERVAL '24 hours'
ORDER BY cart.UpdateDate DESC;

```

--Find which products have never been ordered

--(helps identify potentially problematic products):

```

SELECT
    p.ProductID,
    p.ProductName,
    p.Brand,
    sp.Price,
    sp.Stock,
    s.ShopName
FROM PRODUCT p
JOIN SHOPPRODUCT sp ON p.ProductID = sp.ProductID
JOIN SHOP s ON sp.ShopID = s.ShopID
WHERE p.ProductID NOT IN (
    SELECT DISTINCT ProductID
    FROM ORDERITEM
)
ORDER BY sp.Price ASC;

```

-- Critical shipping analysis and delivery performance:

```

SELECT
    sh.Carrier,
    COUNT(DISTINCT so.ShipmentID) as total_shipments,
    ROUND(AVG(EXTRACT(EPOCH FROM (sh.Est_Delivery_Date -
o.OrderedDate))/86400)) as avg_delivery_days,
    SUM(CASE

```

```

        WHEN sh.ShipStatus = 'Delayed' THEN 1
        ELSE 0
    END) as delayed_shipments,
    w.WarehouseType,
    w.Location as Departure_Location ,
    a. City as Delivery_City

FROM SHIPMENT sh
JOIN SHIPMENTORGANIZATION so ON sh.ShipmentID = so.ShipmentID
JOIN "ORDER" o ON so.OrderID = o.OrderID
JOIN WAREHOUSE w ON sh.WarehouseID = w.WarehouseID
JOIN ADDRESS a ON so.AddressID = a.AddressID
WHERE o.OrderedDate >= CURRENT_DATE - INTERVAL '90 days'
GROUP BY sh.Carrier, w.WarehouseType, a.City,w.Location
ORDER BY carrier,total_shipments DESC;

```